Algebraic Data Types

CSC345: Programming Languages and Paradigms

WCU
WEST CHESTER
UNIVERSITY

Introduction

- *Last class:* type synonyms
  - created names for already existing types using the keyword `type`
  - Also explored parameterized type synonyms
- *Today:* algebraic data types
  - **creating a completely new type** using the keyword `data`

- *From the Haskell Standard Prelude:*

```
data Bool = False | True
```

  - Bool is comprised of two values, False and True
  - ***Prompt:*** what Java construct is similar?

Example 1

```
data StudentRank = Freshman
                 | Sophomore
                 | Junior
                 | Senior
                 | SuperSenior
  deriving Show
```

New type name must begin with a capital letter

Read as “or”

Five Data Constructors
- (constructor name must begin with a capital letter)

“Magic” – automatically generate default code for converting StudentRank values to Strings, for output

```
data StudentRank = Freshman
                 | Sophomore
                 | Junior
                 | Senior
                 | SuperSenior
  deriving Show
```

Declaring "variables":

```
joe :: StudentRank
joe = Freshman
```

```
students :: [StudentRank]
students = [Freshman, Junior, Junior, Sophomore]
```

Functions:

```
shouldGraduate :: StudentRank -> Bool

shouldGraduate Freshman    = False
shouldGraduate Sophomore   = False
shouldGraduate Junior      = False
shouldGraduate Senior      = True
shouldGraduate SuperSenior = True
```

```
shouldGraduate :: StudentRank -> Bool

shouldGraduate Senior      = True
shouldGraduate SuperSenior = True
shouldGraduate _           = False
```

Example 2

```
type Pos = (Int, Int)
data Move = North | South | East | West
  deriving Show
```

Function that applies a move to a position:

```
move :: Move -> Pos -> Pos

move North (x,y) = (x,y+1)
move South (x,y) = (x,y-1)
move East  (x,y) = (x+1,y)
move West  (x,y) = (x-1,y)
```

```
type Pos = (Int, Int)
data Move = North | South | East | West
  deriving Show
```

Function that applies a list of moves, sequentially, given a starting position:

Function Signature:

```
moves :: [Moves] -> Pos -> Pos
```

Function Definition:

```
type Pos = (Int, Int)
data Move = North | South | East | West
  deriving Show
```

Function that returns the “opposite” move:

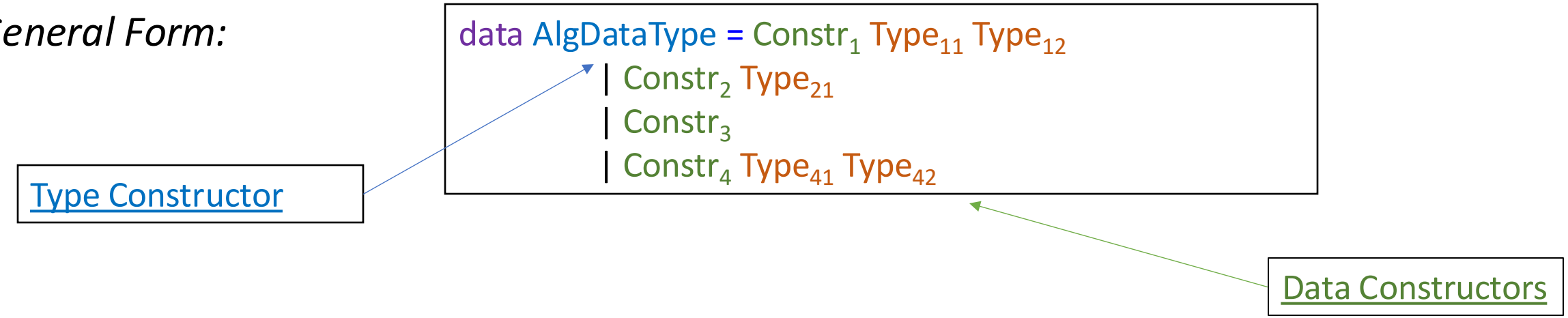
```
rev :: Move -> Move

rev North = South
rev South = North
rev East  = West
rev West  = East
```

Data Constructors with Arguments

- Algebraic data types in Haskell are more powerful than the Java enumeration type!

General Form:



- An algebraic data type has one or more data constructors, and each data constructor can have zero or more arguments

Haskell Naming Rules

- Type and Data Constructor names must always start with a capital letter
- “Variable” and function names must always start with a lowercase letter

Example 1

```
data Shape = Circle Float
           | Rect Float Float
```

- Two constructors, with a different number of arguments.

```
createSquare :: Float -> Shape
createSquare n = Rect n n
```

```
area :: Shape -> Float

(Notice the pattern matching!)
area (Circle r) = pi * r^2
area (Rect l w) = l * w
```

```
isRound :: Shape -> Bool
isRound (Circle _) = True
isRound (Rect _ _) = False
```

```
GHCi> :type Circle
Circle :: Float -> Shape

GHCi> :type Rect
Rect :: Float -> Float -> Shape
```

Example 2

```
data Person = Person String Int
  deriving Show
```

- Common, confusing idiom: type constructor and data constructor are both named `Person`
  - But they are different!
  - Commonly used in the “single constructor” scenario.

```
student1 :: Person
```

```
getAge :: Person -> Int
```

Example 3: the Maybe type constructor

- The following is an example of a parameterized type constructor (that uses the type variable `a`)
- Already declared in the Haskell Standard Prelude:

```
data Maybe a = Nothing
             | Just a
```

- Can be used to avoid exceptions / errors in programming
- Values of type `Maybe a` are values of type `a` that may either FAIL or SUCCEED, as represented by the data constructors
  1. FAIL: Nothing
  2. SUCCEED: Just

- ***Prompt:*** What functions have we seen that could throw an exception?
- New, “safe versions”, that return Nothing, rather than producing an error
- Can be used to avoid exceptions / errors in programming

```
data Maybe a = Nothing
             | Just a
```

```
safehead :: [a] -> Maybe a
```

```
safediv :: Int -> Int -> Maybe Int
```

Conclusion

Advantages of <u>Type Synonyms</u>

- Elements more compact; shorter definitions
- Using tuples (especially pairs) allows the use of polymorphic functions
  - `fst`, `snd`, `zip`, …

Advantages of <u>Algebraic Data Types</u>

- Expressiveness: every object of the type has an explicit label
- Not possible to treat an arbitrary `(String, Int)` as a `Person`
- Easier to understand error messages